Physical AI - Lesson 07: Forward Kinematics

Module 2: Robot Kinematics | Lesson 7 of 40

Status: Python exercise and Isaac Sim visual demonstration completed (learner-reported).

Scope: Planar one-link and two-link forward kinematics; no physics articulation.

Learning objectives

1. Explain forward kinematics (FK): joint configuration and geometry -> end-effector position or pose.

2. Use sine and cosine to locate the tip of a single planar link.

3. Distinguish a relative elbow angle from the second link's world orientation.

4. Calculate two-link tip coordinates and the centers/endpoints of visual link geometry.

5. Compare handwritten results with Python output and a manually arranged Isaac Sim scene.

1. Foundation

Joint space holds joint variables such as q = (theta1, theta2). Cartesian space holds positions such as p = (x, y). For a general 3D manipulator, FK computes end-effector pose (position plus orientation), not only position.

In these exercises, the shoulder is fixed at (0, 0); positive joint angles rotate counterclockwise in the XY plane. The elbow angle theta2 is relative to Link1.

2. One-link FK

For a link of length L and base angle theta:

```
x = L cos(theta)
y = L sin(theta)
```

Examples successfully solved:

- L = 3 m, theta = 90 deg -> (0, 3) m.

- L = 4 m, theta = 60 deg -> (2, 3.464...) m.

3. Two-link FK

Let lengths be L1 and L2. Let theta1 be the world-relative shoulder angle and theta2 be the elbow angle relative to Link1. Then Link2's world orientation is theta1 + theta2.

```
x_e = L1 cos(theta1)
y_e = L1 sin(theta1)
```

```
x_g = L1 cos(theta1) + L2 cos(theta1 + theta2)

y_g = L1 sin(theta1) + L2 sin(theta1 + theta2)
```

For a visually centered bar representing Link2, its center is located halfway from the elbow to the gripper:

```
x_c = x_e + (L2/2) cos(theta1 + theta2)

y_c = y_e + (L2/2) sin(theta1 + theta2)
```

Worked example: L1=3 m, L2=2 m, theta1=0 deg, theta2=60 deg.

- Elbow: (3, 0) m.
- Link2 center: (3.5, 0.8660254...) m.
- Gripper: (4, 1.7320508...) m.

The learner manually solved x=4 and y=1.732, then ran a Python script that printed 4.000 and 1.732 meters.

4. Isaac Sim 6.1 visual experiment

This is a geometric visualization, not a simulated articulated robot. The two USD Cube prims were placed separately under World. Each Cube has `size = 1`, and the stage metadata was observed as `metersPerUnit = 1` in a saved USDA file.

```
| Property | Link1 | Link2 |

|---|---|---|

| USD prim | Cube | Cube |

| Size | 1 | 1 |

| Translate | (1.5, 0, 0) | (3.5, 0.866, 0) |

| Rotate XYZ | (0, 0, 0) deg | (0, 0, 60) deg |

| Scale | (3, 0.2, 0.2) | (2, 0.2, 0.2) |

| Nominal dimensions | 3 x 0.2 x 0.2 m | 2 x 0.2 x 0.2 m |
```

The learner reported visually observing Link1 and Link2 meeting at the elbow, then saving the scene as `lesson_07_forward_kinematics.usda`.

Geometric verification: A centered unit Cube scaled to length 3 and translated to x=1.5 extends from x=0 to x=3. The second scaled Cube has half-length 1; translating to (3.5, 0.866, 0) and rotating +60 deg around Z places its nominal endpoints at approximately (3, 0, 0) and (4, 1.732, 0).

Precision: The scene used a rounded Y translation of 0.866 rather than sqrt(3)/2 = 0.8660254..., so these endpoints are approximate. The geometric endpoints are not physical joints or an actual gripper object.

5. Troubleshooting and environment observations

Isaac Sim's Script Editor failed even for `print("Hello Isaac Sim")`, reporting an inability to open its temporary script file. Windows File Explorer opened `%LOCALAPPDATA%\Temp`, but the specific temporary subdirectory from an error message was missing when checked. The root cause was not established. Stage units were instead verified by saving and reading the ASCII USDA file. This Script Editor fault does not invalidate the separate Windows Python FK output.

6. Reproduce and test

Use Python 3's standard library; Isaac Sim is not required for these checks:

```
python lesson_07_fk_checks.py
```

The script calculates one- and two-link positions, prints elbow/center/gripper coordinates, and checks the expected values using tolerances. It does not open or inspect the saved USDA scene and does not control a robot.

7. Knowledge checks and reflections

1. Why does a two-link arm use `theta1 + theta2` for Link2's orientation?
2. If only theta2 changes, why does the elbow position stay fixed?
3. Why must the visual Link2 Cube be translated to the link's midpoint rather than to the elbow?
4. What extra data would you need to compute a 3D robot's complete gripper pose?
5. Why is a visually connected pair of Cube prims not equivalent to a physics articulation?

8. Evidence and limitations

See `experiment_evidence.md` for verified-by-user observations, derived mathematical results, unresolved issues and evidence gaps. The USD/USDA scene was reported as saved locally on the learner's PC but was not uploaded or independently inspected here. No screenshot of the final two-link scene or direct simulator measurement of the endpoint was supplied. Do not equate this lesson's simplified 2D arm with Franka robot kinematics.

Next lesson: Lesson 8 has not started.